AMS597Project

Zian Shang, Nicole Dona, Thomas Huang, Tasfia Shaikh

2025-04-16

Install + Include packages

```
library(mice)
```

```
##
## Attaching package: 'mice'

## The following object is masked from 'package:stats':
##
##     filter

## The following objects are masked from 'package:base':
##
##     cbind, rbind
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v readr     2.1.5
## v ggplot2   3.5.1     v stringr   1.5.1
## v lubridate 1.9.4     v tibble    3.2.1
## v purrr     1.0.4     v tidyr     1.3.1
```

```
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x dplyr::filter() masks mice::filter(), stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(mlbench)
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(e1071)
library(logistf)
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:ggplot2':
##
##     margin
##
## The following object is masked from 'package:dplyr':
##
##     combine
```

```
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
library(clustMixType)
library(cluster)
library(MASS)
```

```
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##     select
```

```
library(ggplot2)
library(tidyr)
library(rpart)
library(rattle)
```

```
## Loading required package: bitops
## Rattle: A free graphical interface for data science with R.
## Version 5.5.1 Copyright (c) 2006-2021 Togaware Pty Ltd.
## Type 'rattle()' to shake, rattle, and roll your data.
##
## Attaching package: 'rattle'
##
## The following object is masked from 'package:randomForest':
##
##     importance
```

```
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following object is masked from 'package:bitops':
##
##     %&%
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-8
```

Read data from "chronic_kidney_disease.arff" file

```
# change to your own file path
file_path <- "Chronic_Kidney_Disease/chronic_kidney_disease.arff"
lines <- readLines(file_path)


# get lines after line "@data" all the way to the end
data_start <- which(grepl("^@data", lines, ignore.case = TRUE)) + 2
data_lines <- lines[data_start:length(lines)-1]

rm(file_path, data_start, lines)
```

```
# read data lines as table
data <- read.table(text = data_lines,
                   sep = ",",
                   header = FALSE,
                   stringsAsFactors = FALSE,
                   na.strings = "?")

# rename columns
colnames(data) <- c("age", "bp", "sg", "al", "su", "rbc", "pc", "pcc", "ba", "bgr",
                    "bu", "sc", "sod", "pot", "hemo", "pcv", "wbcc", "rbcc", "htn",
                    "dm", "cad", "appet", "pe", "ane", "class")
```

```r
# str(data)
# dim(data)    400x25

rm(data_lines)
```

Explore columns that might be useful for modeling

```r
# Count number of NA's in each column
na.sums <- colSums(is.na(data))  # Now check NA counts

# print num of cols that has NA's > 50% of total
sum(na.sums > (0.5 * nrow(data)))
```

```
## [1] 0
```

```r
rm(na.sums)

# All columns have missing values less than 50% of the total data,
# impute missing values instead.

# lapply(data, unique)
```

Reformat strings

```r
data[] <- lapply(data, function(x) {
  # for every char col
  if (is.character(x)){
    x <- gsub("\\t\\?", NA, x)      # convert "\t?" to NA
    x <- gsub("^\\t", "", x)        # remove leading "\t" for some strings
  }
  return(x)
})

# dm
data$dm[data$dm == " yes"] <- "yes"

# ckd
data$class[data$class == "ckd\t"] <- "ckd"

# check unique values
# lapply(data, unique)
```

Check & deal with data types

```r
print(sapply(data, class))
```

```
##         age          bp          sg          al          su         rbc
##   "integer"   "integer"   "numeric"   "integer"   "integer" "character"
##          pc         pcc          ba         bgr          bu          sc
## "character" "character" "character"   "integer"   "numeric"   "numeric"
##         sod         pot        hemo         pcv        wbcc        rbcc
##   "numeric"   "numeric"   "numeric" "character" "character" "character"
##         htn          dm         cad       appet          pe         ane
## "character" "character" "character" "character" "character" "character"
##       class
## "character"
```

```r
# Factorize numeric but categorical data
cols1 <- c("sg", "al", "su")
# Apply factorization to selected columns using lapply
data[cols1] <- lapply(data[cols1], as.factor)

cols2 <- c("pcv", "wbcc", "rbcc")
data[cols2] <- lapply(data[cols2], as.numeric)

rm(cols1, cols2)
```

Impute numeic data by pmm, categorical data by mode

```r
# Function to calculate the mode
get.mode <- function(x) {
  unique.values <- na.omit(x)
  mode <- unique.values[which.max(tabulate(match(x, unique.values)))]
  return(mode)
}
```

```r
# Function to impute missing values
impute.data <- function(df) {
  # Separate numeric and categorical columns
  num_cols <- names(df)[sapply(df, is.numeric)]
  cat_cols <- names(df)[sapply(df, function(x) is.character(x) || is.factor(x))]

  # PMM for numeric columns (only if there are missing values)
  if (any(colSums(is.na(df[num_cols])) > 0)) {
    set.seed(597)
    temp <- mice(df[num_cols], method = "pmm", m = 1, maxit = 5, printFlag = FALSE)
    df[num_cols] <- complete(temp)
  }

  # Mode Imputation for categorical columns
  for (col in cat_cols) {
    df[[col]][is.na(df[[col]])] <- get.mode(df[[col]])
  }

  return(df)
}
```

```r
# Apply the function
data.imputed <- impute.data(data)

colSums(is.na(data.imputed))
```

```
##   age    bp    sg    al    su   rbc    pc   pcc    ba   bgr    bu    sc   sod
##     0     0     0     0     0     0     0     0     0     0     0     0     0
##   pot  hemo   pcv  wbcc  rbcc   htn    dm   cad appet    pe   ane class
##     0     0     0     0     0     0     0     0     0     0     0     0
```

factorize data

```r
# data with char columns factored
data.factorize <- data.imputed

# change class values to "1" & "0"
data.factorize$class <- ifelse(data.factorize$class == "ckd", "1", "0")


# factorize char columns
data.factorize[] <- lapply(data.factorize, function(x){
  if(is.character(x)){ x <- as.factor(x) }
  return(x)
})

# str(data.factorize)
```

detect & fix outliers

```r
# function to detect outlier columns
get_outliers <- function(df) {
  boxplot_stats <- lapply(df, function(x) {
    if (is.numeric(x)) boxplot.stats(x)$out else NULL
  })

  outlier_cols <- names(boxplot_stats)[sapply(boxplot_stats, function(x) !is.null(x) && length(x) > 0)]
  return(outlier_cols)
}
get_outliers(data.factorize)
```

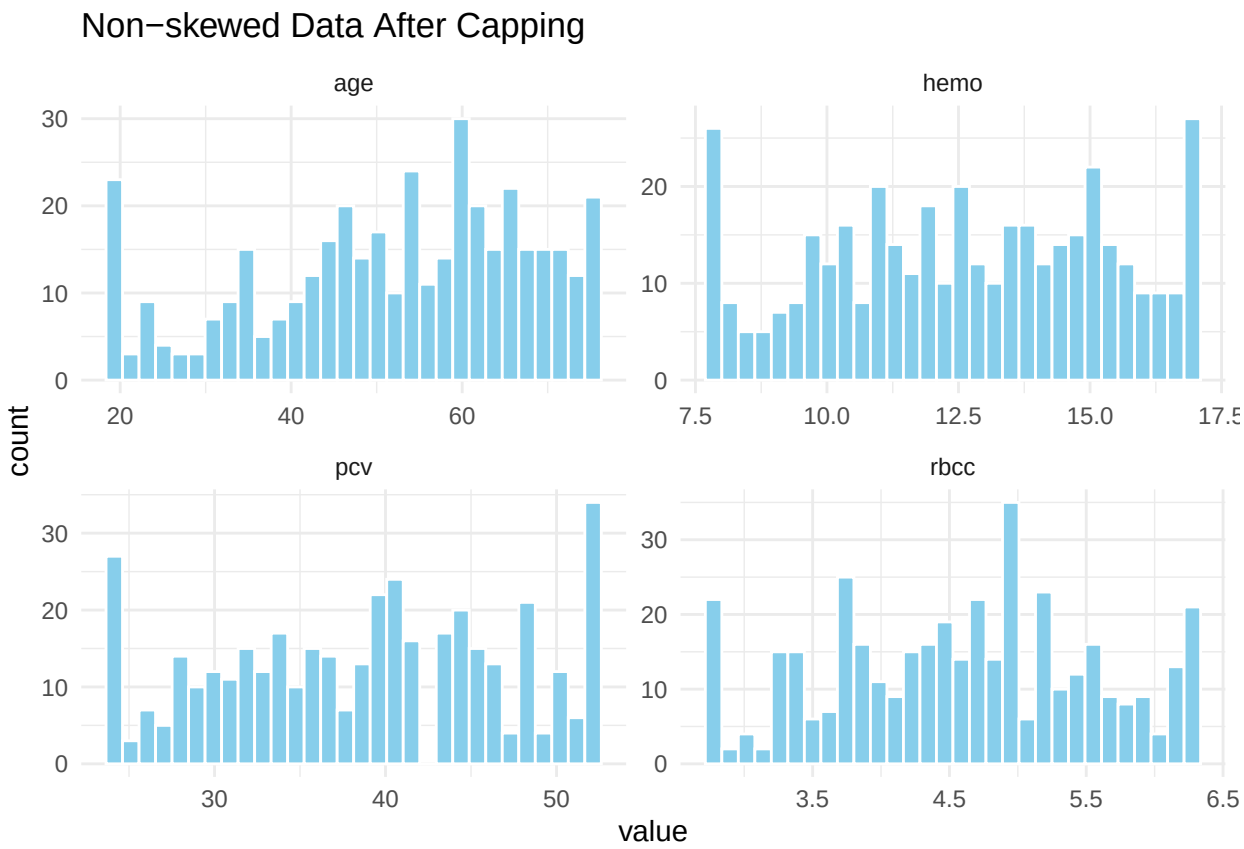
```
##  [1] "age"  "bp"   "bgr"  "bu"   "sc"   "sod"  "pot"  "hemo" "pcv"  "wbcc"
## [11] "rbcc"
```

```r
# identify skewed columns
# sapply(data.factorize[outlier_cols], skewness, na.rm = TRUE)

# bp, bgr, bu, sc, pot, wbcc -- skewness > 1, highly skewed
# age, hemo, pcv, rbcc       --  -1 < skewness < 1
```

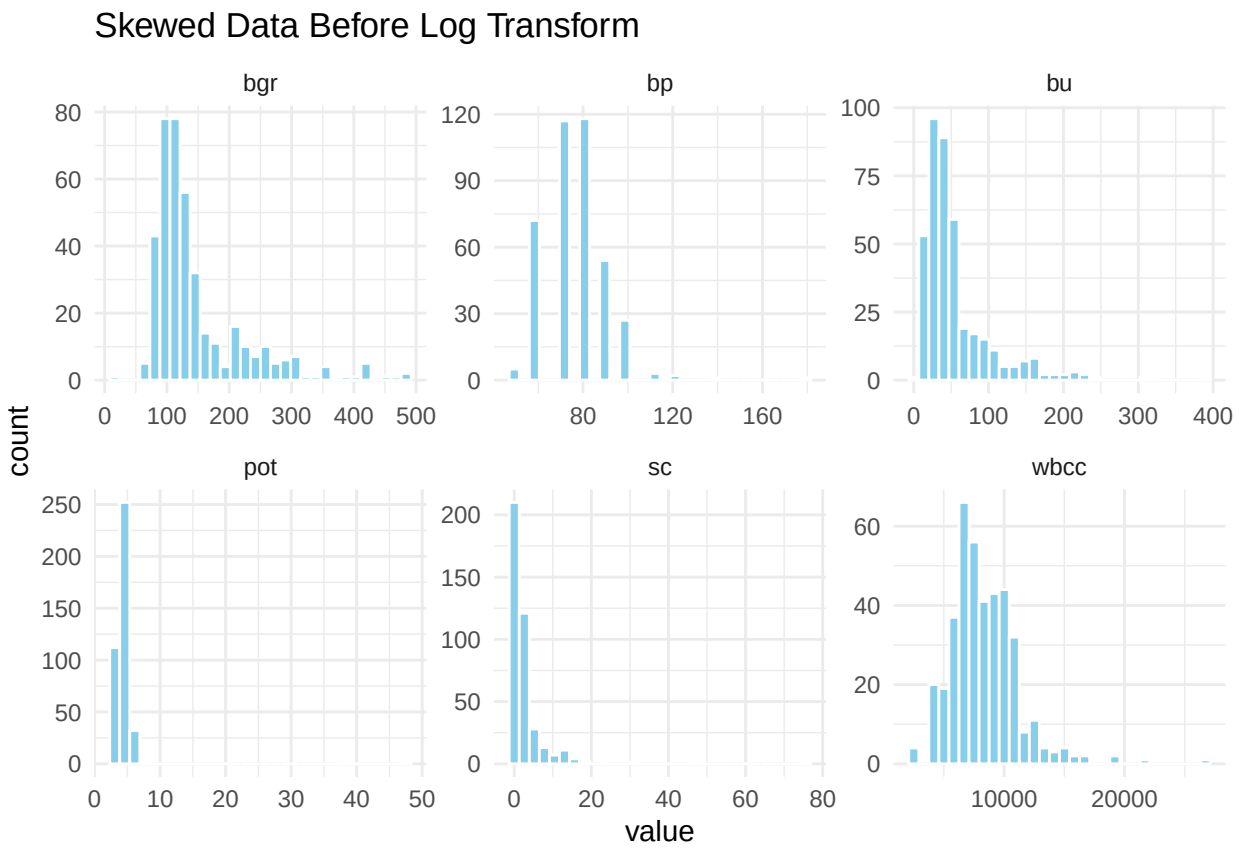
```
# sod                          -- skewness <-1, strongly negative skew

skewed.cols <- c("bp", "bgr", "bu", "sc", "pot", "wbcc")
not.skewed.cols <- c("age", "hemo", "pcv", "rbcc")  # More stable data
```

transformation for skewed data/extreme values

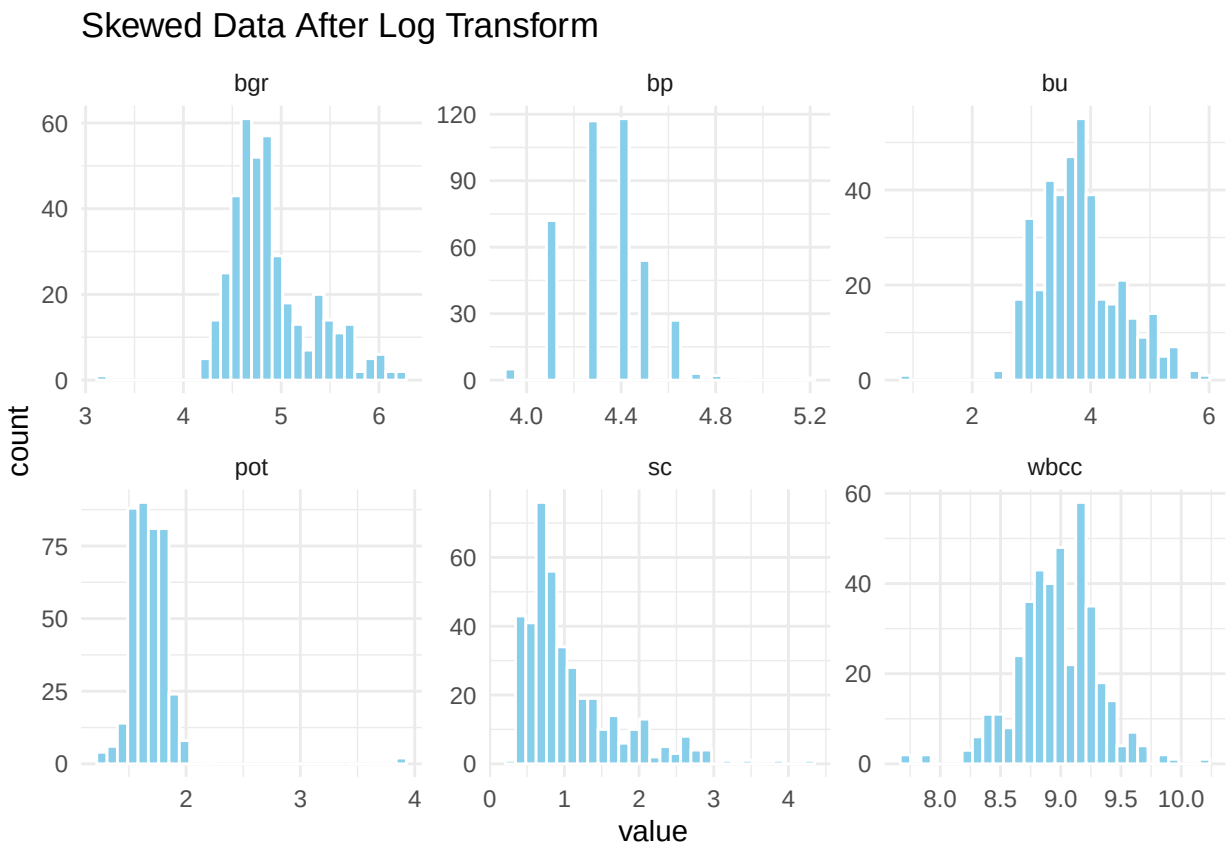
```
# 1. capping for not highly skewed data with extreme values

# "age", "hemo", "pcv", "rbcc" --> solved
cap_outliers <- function(x) {
  qnt <- quantile(x, probs = c(0.05, 0.95), na.rm = TRUE)
  x[x < qnt[1]] <- qnt[1]
  x[x > qnt[2]] <- qnt[2]
  return(x)
}
data.factorize[not.skewed.cols] <- lapply(data.factorize[not.skewed.cols], cap_outliers)

data.factorize[not.skewed.cols] %>%
  pivot_longer(everything(), names_to = "variable") %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "skyblue", color = "white") +
  facet_wrap(~ variable, scales = "free") +
  theme_minimal() +
  labs(title = "Non-skewed Data After Capping")
```




```
data.factorize[skewed.cols] %>%
  pivot_longer(everything(), names_to = "variable") %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "skyblue", color = "white") +
  facet_wrap(~ variable, scales = "free") +
  theme_minimal() +
  labs(title = "Skewed Data Before Log Transform")
```

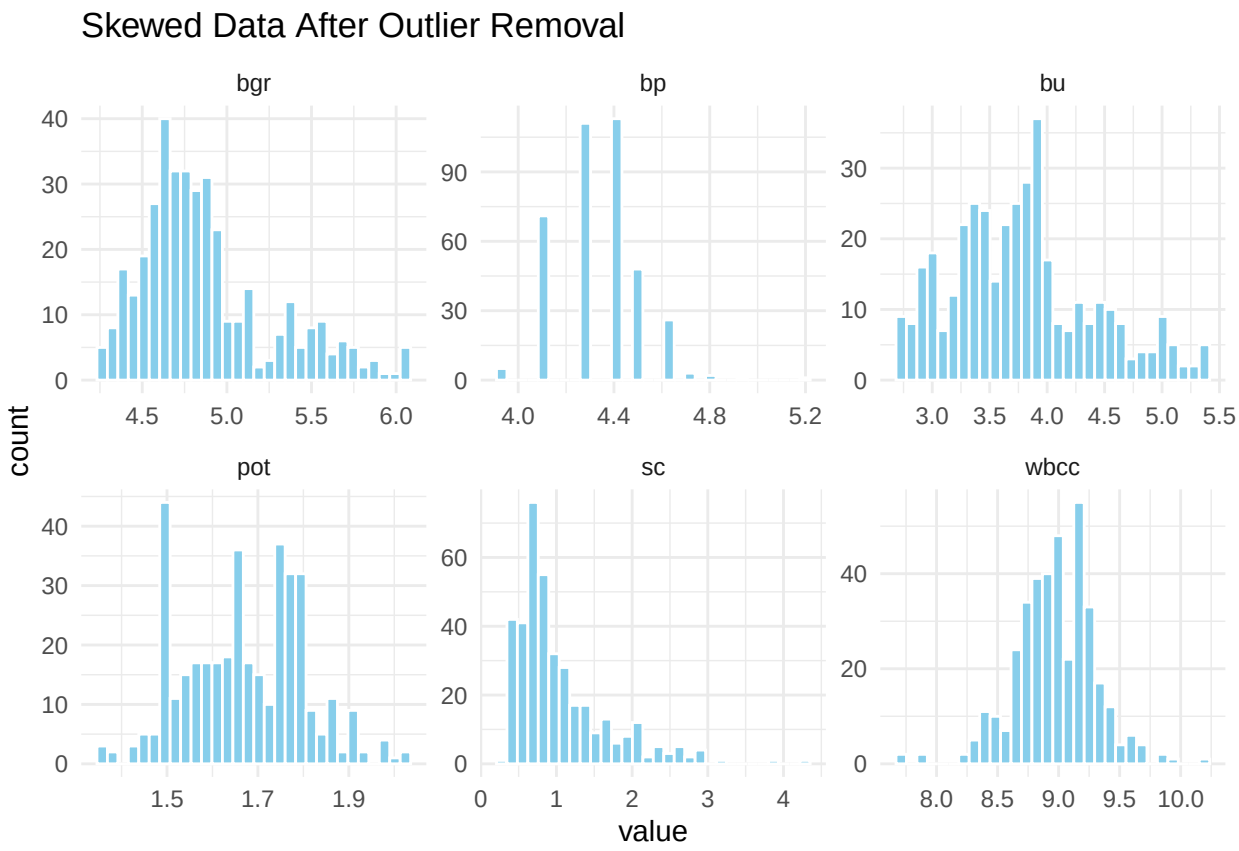



```
# ------------------------------------------------------------------------------

# 2. log-transform highly skewed data --> reduce skewness

data.factorize[skewed.cols] <- log1p(data.factorize[skewed.cols])
## need to un-log it for k-means

# ------------------------------------------------------------------------------

data.factorize[skewed.cols] %>%
  pivot_longer(everything(), names_to = "variable") %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "skyblue", color = "white") +
  facet_wrap(~ variable, scales = "free") +
  theme_minimal() +
  labs(title = "Skewed Data After Log Transform")
```

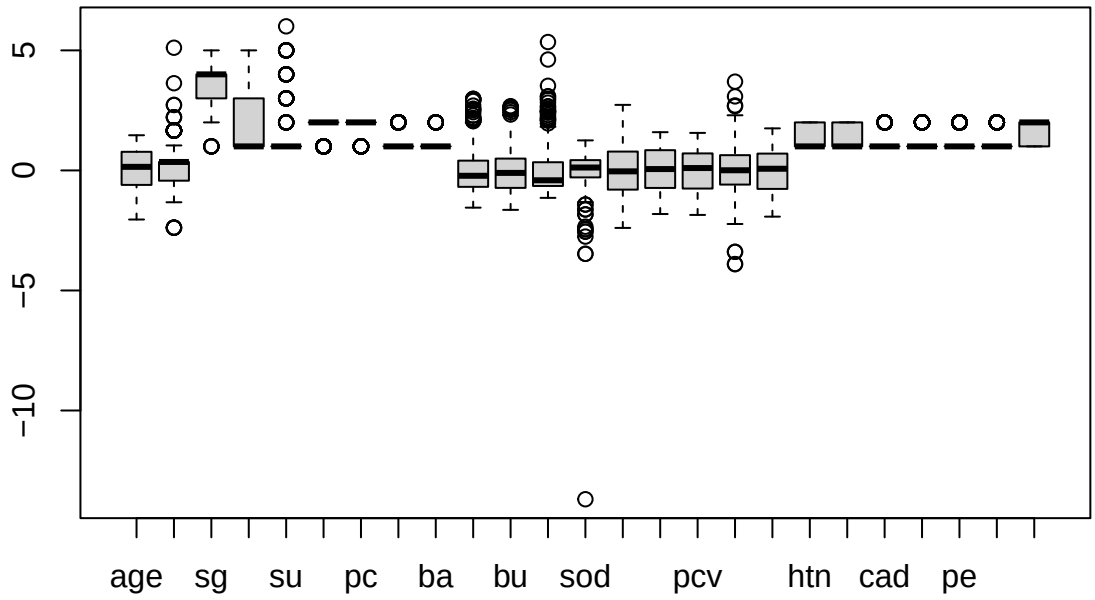


According to the histograms of skewed data after log-transformation above, some outliers may be meaningful, so we will only remove outlier in pot, bu, bgr.

```
# 2. remove outliers for pot, bu, bgr
cols <- c("pot", "bu", "bgr")

for (col in cols) {
  q <- quantile(data.factorize[[col]], probs = c(0.01, 0.99), na.rm = TRUE)
  data.factorize <- data.factorize[data.factorize[[col]] >= q[1] & data.factorize[[col]] <= q[2], ]
}

# nrow(data.factorize)     --> 381 rows left

# chcek removal result with plot
data.factorize[skewed.cols] %>%
  pivot_longer(everything(), names_to = "variable") %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "skyblue", color = "white") +
  facet_wrap(~ variable, scales = "free") +
  theme_minimal() +
  labs(title = "Skewed Data After Outlier Removal")
```




```
# ------------------------------------------------------------------------------

# 3. Standardize/scale the numeric columns
numeric.cols <- sapply(data.factorize, is.numeric)
scaled_numeric <- scale(data.factorize[, numeric.cols])
data.factorize[, numeric.cols] <- scaled_numeric

# record means and sds for k-means clustering
means <- attr(scaled_numeric, "scaled:center")
sds <- attr(scaled_numeric, "scaled:scale")

boxplot(data.factorize)
```




```
# test outliers again in the end
get_outliers(data.factorize)
```

```
## [1] "bp"   "bgr"  "bu"   "sc"   "sod"  "wbcc"
```

```
# meaningful outliers is fine

rm(not.skewed.cols)
```

Feature selection

```
# 1. data as Matrix
X <- as.matrix(subset(data.factorize, select=-c(class)))
Y <- data.factorize$class

# ------------------------------------------------------------------------------

# 2. split data
set.seed(597)
training.samples <- data.factorize$class %>% createDataPartition(p = 0.8, list = F)
train.data <- data.factorize[training.samples, ]
test.data <- data.factorize[-training.samples,]
```

```r
X_train <- X[training.samples,]
X_test <- X[-training.samples,]
y_train <- Y[training.samples]
y_test <- Y[-training.samples]

# ------------------------------------------------------------------------

# 3. Variable Selection -> ElasticNet
# reasons:
# 1. To remove the variables that have little or no assoication with the target,
#    to make it more easy to interpret.
# 2. If features are highly correlated (e.g., two similar lab tests), ElasticNet
#    tends to keep one and remove others.

train.data$class <- factor(ifelse(train.data$class == "1", "ckd", "notckd"),
                           levels = c("notckd", "ckd"))

# define cross-validation control
ctrl <- trainControl(
  method = "cv",
  number = 10,
  classProbs = T,
  summaryFunction = twoClassSummary,
  verboseIter = F
)

set.seed(123)

model <- train(class ~ ., data = train.data, method = "glmnet",
               trControl = ctrl, metric = "ROC")

# ------------------------------------------------------------------------

# 3. get model with best tune
# model$bestTune -> gives best tune value
suppressWarnings({
  model_en <- glmnet( x = X_train, y = y_train,
                      alpha = model$bestTune$alpha,
                      lambda=model$bestTune$lambda,
                      family = "binomial")

})

# ------------------------------------------------------------------------

# get model coefficients
coef_df <- as.matrix(coef(model_en))
coef_df <- data.frame(name = rownames(coef_df), coef = coef_df[,1])

# obtain variables to keep
selected_vars <- coef_df$name[coef_df$coef != 0 & coef_df$name != "(Intercept)"]
data.factorize <- data.factorize[, c(selected_vars, "class")]
```

```r
str(data.factorize)
```

```
## 'data.frame':    381 obs. of  14 variables:
##  $ age  : num  -0.228 -2.045 0.649 -0.04 0.524 ...
##  $ bp   : num  0.352 -2.386 0.352 0.352 1.041 ...
##  $ sg   : Factor w/ 5 levels "1.005","1.01",..: 4 4 2 2 3 2 3 3 4 2 ...
##  $ al   : Factor w/ 6 levels "0","1","2","3",..: 2 5 3 3 4 1 3 4 3 4 ...
##  $ su   : Factor w/ 6 levels "0","1","2","3",..: 1 1 4 1 1 1 5 1 1 1 ...
##  $ bgr  : num  -0.182 -0.66 2.969 -0.514 -1.413 ...
##  $ bu   : num  -0.271 -1.362 0.347 -0.787 -0.849 ...
##  $ sc   : num  -0.4109 -0.7359 -0.0204 -0.27 -0.4863 ...
##  $ sod  : num  0.223 0.223 -1.112 1.25 0.428 ...
##  $ pot  : num  -0.4867 0.6533 0.5197 -0.0391 -1.8259 ...
##  $ hemo : num  0.993 -0.543 -1.18 -0.431 -0.206 ...
##  $ pcv  : num  0.586 -0.146 -1 -0.512 -0.024 ...
##  $ rbcc : num  0.595 -0.455 0.175 -0.035 -0.245 ...
##  $ class: Factor w/ 2 levels "0","1": 2 2 2 2 2 2 2 2 2 2 ...
```

```r
# update skew.cols
skewed.cols <- c("bp", "bgr", "bu", "sc", "pot")
not.skewed.cols <- c("age", "hemo", "pcv", "rbcc")

rm(X, Y, X_train, X_test, y_train, y_test, ctrl, model, coef_df, selected_vars)
```

Research Question 1:

Can we predict whether a patient has chronic kidney disease based on clinical and laboratory features?

1. logistic model

```r
# split data
set.seed(597)
training.samples <- data.factorize$class %>% createDataPartition(p=0.8, list=F)
train.data <- data.factorize[training.samples,]
test.data <- data.factorize[-training.samples,]
```

```r
# fit model

# 1. data does not converge
# fit.logistic <- glm(class~., train.data, family=binomial)

# 2. force convergence by controlling the maxit & add class weights
suppressWarnings({
set.seed(597)
fit.logistic <- glm(class ~ ., data = train.data,
                              family = binomial,
                              weights = ifelse(train.data$class == 1, 1, 10),
                              control = glm.control(maxit = 100))
})
```

```r
# extreme 0/1 warning persists, NOT imbalance class problem


# 3. try stepwise selection, 0.925 accuracy
suppressWarnings({
  set.seed(597)
fit.logistic <- step(fit.logistic, direction = "both", trace = 0)
})
summary(fit.logistic)
```

```
##
## Call:
## glm(formula = class ~ sg + al + su + bgr + sc + sod + pot + pcv,
##     family = binomial, data = train.data, weights = ifelse(train.data$class ==
##         1, 1, 10), control = glm.control(maxit = 100))
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.840e+03  3.147e+06   0.001    1.000
## sg1.01       1.182e+03  3.176e+06   0.000    1.000
## sg1.015     -2.535e+02  8.421e+06   0.000    1.000
## sg1.02      -1.100e+03  3.047e+06   0.000    1.000
## sg1.025     -1.743e+03  3.137e+06  -0.001    1.000
## al1          1.882e+03  1.684e+06   0.001    0.999
## al2          5.876e+02  7.973e+06   0.000    1.000
## al3          2.201e+03  1.734e+06   0.001    0.999
## al4          2.336e+03  8.102e+06   0.000    1.000
## su1         -2.474e+03  8.466e+06   0.000    1.000
## su2          1.938e+03  2.058e+06   0.001    0.999
## su3         -1.038e+03  2.936e+06   0.000    1.000
## su4         -7.568e+01  8.189e+06   0.000    1.000
## su5         -1.890e+03  6.759e+07   0.000    1.000
## bgr          8.490e+02  4.886e+05   0.002    0.999
## sc           1.945e+03  1.101e+06   0.002    0.999
## sod          6.321e+02  3.644e+05   0.002    0.999
## pot         -5.215e+02  3.018e+05  -0.002    0.999
## pcv         -1.083e+03  6.373e+05  -0.002    0.999
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1.0941e+03  on 305  degrees of freedom
## Residual deviance: 2.5180e-09  on 287  degrees of freedom
## AIC: 38
##
## Number of Fisher Scoring iterations: 36
```

```r
# extreme 0/1 warning
```

test logistic model prediction

```r
set.seed(597)
# predict
pred_prob <- fit.logistic %>% predict(test.data, type="response")
predicted.classes <- ifelse(pred_prob > 0.5, "1", "0")

# confusion matrix
cm.l <- confusionMatrix(factor(predicted.classes), factor(test.data$class), positive = "1")
cm.l
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 29  2
##          1  0 44
##
##                Accuracy : 0.9733
##                  95% CI : (0.907, 0.9968)
##     No Information Rate : 0.6133
##     P-Value [Acc > NIR] : 1.375e-13
##
##                   Kappa : 0.9445
##
##  Mcnemar's Test P-Value : 0.4795
##
##             Sensitivity : 0.9565
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.9355
##              Prevalence : 0.6133
##          Detection Rate : 0.5867
##    Detection Prevalence : 0.5867
##       Balanced Accuracy : 0.9783
##
##        'Positive' Class : 1
##
```

Extract Metrics

```r
accuracy <- cm.l$overall["Accuracy"]
sensitivity <- cm.l$byClass["Sensitivity"]
specificity <- cm.l$byClass["Specificity"]

cat("Accuracy: ", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy:  97.33 %
```

```r
cat("Sensitivity: ", round(sensitivity * 100, 2), "%\n")
```

```
## Sensitivity:  95.65 %
```

```r
cat("Specificity: ", round(specificity * 100, 2), "%\n")
```

```
## Specificity:  100 %
```

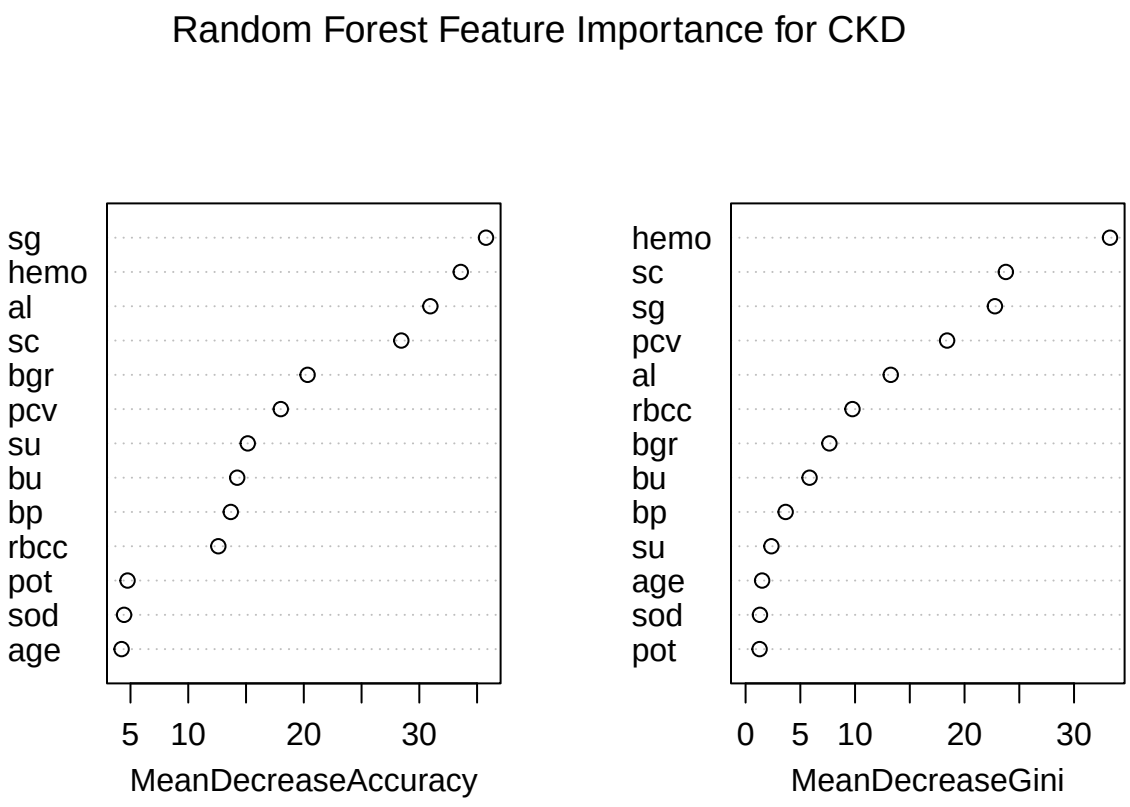
Conclusion for logistic model: The best-performing logistic regression model achieved a relatively high accuracy after applying maximum iterations (maxit), class weighting, and step-wise variable selection. Despite these adjustments, the warning "glm.fit: fitted probabilities numerically 0 or 1 occurred" persisted, indicating potential issues with extreme values in the dataset. While this warning doesn't suggest an imbalance in the classes, further investigation into the dataset's characteristics would be necessary to address this issue, but it remains beyond the scope of the current analysis. Next, we will explore using a random forest model to assess its performance and potential to handle these challenges.

2. RandomForest

```r
# Train Random Forest
set.seed(597)
fit.rf <- randomForest(class ~ ., data = train.data, importance = TRUE, ntree = 500)
```

Plot variable importance

```r
varImpPlot(fit.rf, main = "Random Forest Feature Importance for CKD")
```



Get importance table

```
importance_df <- as.data.frame(fit.rf$importance)
importance_df <- importance_df[order(importance_df$MeanDecreaseGini, decreasing = TRUE), ]
print(importance_df)
```

```
##                0             1 MeanDecreaseAccuracy MeanDecreaseGini
## hemo 0.299273413  0.0222655345          0.128758102        33.287237
## sc   0.222010587  0.0095355597          0.092070614        23.773795
## sg   0.259518256  0.0157419488          0.109980414        22.772936
## pcv  0.139850789  0.0074604500          0.058800774        18.406381
## al   0.226363897  0.0046958866          0.090140194        13.259569
## rbcc 0.071870595  0.0040007637          0.030397170         9.762035
## bgr  0.108117726  0.0002744237          0.041924235         7.656533
## bu   0.079802703  0.0030581033          0.032577305         5.842991
## bp   0.057595895  0.0004098372          0.022876120         3.658867
## su   0.040950972  0.0001306820          0.015945304         2.358306
## age  0.003179579  0.0005511302          0.001552763         1.511555
## sod  0.008301058 -0.0003007632          0.002999801         1.313340
## pot  0.005607306  0.0005370118          0.002480548         1.279333
```

```
rm(importance_df)
```

Predict model & analyze

```
# get model predictions
rf_predictions <- predict(fit.rf, newdata = test.data)

# Create confusion matrix using caret
cm.f <- confusionMatrix(rf_predictions, test.data$class , positive="1")
cm.f
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 29  1
##          1  0 45
##
##                Accuracy : 0.9867
##                  95% CI : (0.9279, 0.9997)
##     No Information Rate : 0.6133
##     P-Value [Acc > NIR] : 5.768e-15
##
##                   Kappa : 0.9721
##
##  Mcnemar's Test P-Value : 1
##
##             Sensitivity : 0.9783
##             Specificity : 1.0000
```

```
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.9667
##              Prevalence : 0.6133
##          Detection Rate : 0.6000
##    Detection Prevalence : 0.6000
##       Balanced Accuracy : 0.9891
##
##        'Positive' Class : 1
##
```

Extract Metrics

```
accuracy <- cm.f$overall["Accuracy"]
sensitivity <- cm.f$byClass["Sensitivity"]
specificity <- cm.f$byClass["Specificity"]

cat("Accuracy: ", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy:  98.67 %
```

```
cat("Sensitivity: ", round(sensitivity * 100, 2), "%\n")
```

```
## Sensitivity:  97.83 %
```

```
cat("Specificity: ", round(specificity * 100, 2), "%\n")
```

```
## Specificity:  100 %
```

logistic: Accuracy: 98.67 % , Sensitivity: 97.83 % , Specificity: 100 %

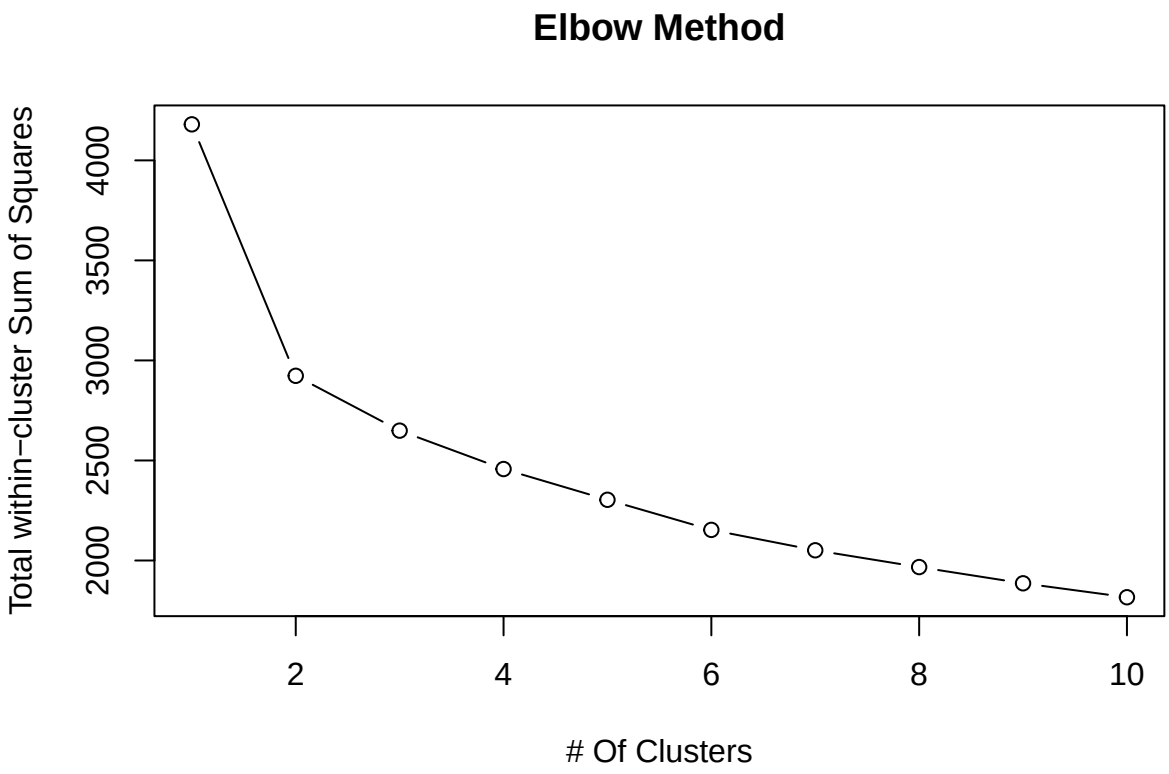
Conclusion for randomForest model: High accuracy, sensitivity, and specificity, better than logistic model and without warning. Both are good at predicting non-CKD objects.

Research Question 2:

Can we identify distinct subgroups of patients based on their clinical and laboratory features?

Decide the number of clusters, use the elbow method

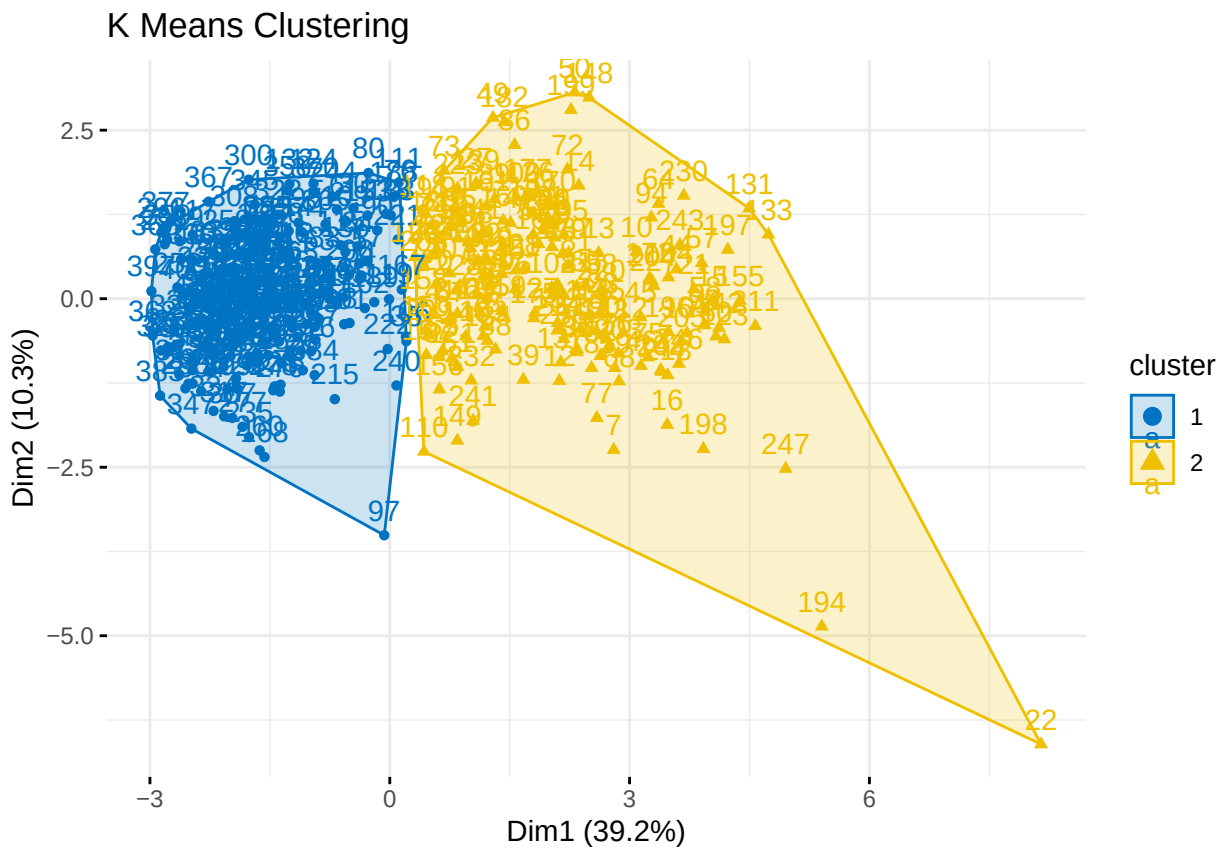
```
wss <- numeric(10)
for (k in 1:10){
  set.seed(597)
  kmeans_result <- kmeans(scaled_numeric, centers=k, nstart=50)
  wss[k] <- kmeans_result$tot.withinss
}

plot(1:10, wss, type='b', main='Elbow Method', xlab= '# Of Clusters',
     ylab='Total within-cluster Sum of Squares',)
```




```
#### Choose 2 clusters


rm(kmeans_result)
```

1. K-means clustering

```
set.seed(597)
km <- kmeans(scaled_numeric, centers=2, nstart=50)
fviz_cluster(km, data=scaled_numeric, ellipse.type='convex', palette='jco',
             ggtheme=theme_minimal(), main='K Means Clustering')
```



calculate centroids

```
# match column order in sds & means with that of model centers
sds <- sds[colnames(km$centers)]
means <- means[colnames(km$centers)]

# get centroids
centroids <- sweep(km$centers, 2, sds, "*")
centroids <- sweep(centroids, 2, means, "+")
# de-log skewed columns
centroids[ , skewed.cols] <- expm1(centroids[ , skewed.cols])

# print centroids in dataframe
centroids <- round(centroids, 2)
centroids <- as.data.frame(centroids)
centroids$Cluster <- paste("Cluster", 1:nrow(centroids))
centroids <- centroids[, c(ncol(centroids), 1:(ncol(centroids)-1))]
print(centroids)
```

```
##     Cluster   age    bp    bgr    bu   sc    sod  pot  hemo   pcv wbcc rbcc
## 1 Cluster 1 45.82 72.20 111.05 29.93 1.00 140.95 4.22 14.54 44.71 8.93 5.24
## 2 Cluster 2 59.51 79.76 161.09 68.62 3.56 133.61 4.47 10.33 31.75 9.03 3.82
```

```
# cluster 1 is CKD  -> age around 45
# cluster 2 is not CKD -> age around 60
```

```
# Compare with data before factorization to ensure values make sense
summary(data.imputed[numeric.cols])
```

```
##       age              bp              bgr              bu
##  Min.   : 2.00   Min.   : 50.00   Min.   : 22.0   Min.   :  1.50
##  1st Qu.:42.00   1st Qu.: 70.00   1st Qu.: 99.0   1st Qu.: 27.00
##  Median :55.00   Median : 80.00   Median :120.0   Median : 41.00
##  Mean   :51.63   Mean   : 76.62   Mean   :147.3   Mean   : 56.55
##  3rd Qu.:64.25   3rd Qu.: 80.00   3rd Qu.:162.2   3rd Qu.: 64.25
##  Max.   :90.00   Max.   :180.00   Max.   :490.0   Max.   :391.00
##        sc              sod              pot             hemo
##  Min.   : 0.400   Min.   :  4.5   Min.   : 2.500   Min.   : 3.10
##  1st Qu.: 0.900   1st Qu.:135.0   1st Qu.: 3.800   1st Qu.:10.47
##  Median : 1.200   Median :139.0   Median : 4.300   Median :12.65
##  Mean   : 3.003   Mean   :137.7   Mean   : 4.665   Mean   :12.59
##  3rd Qu.: 2.800   3rd Qu.:142.0   3rd Qu.: 4.900   3rd Qu.:15.00
##  Max.   :76.000   Max.   :163.0   Max.   :47.000   Max.   :17.80
##       pcv             wbcc            rbcc
##  Min.   : 9.00   Min.   : 2200   Min.   :2.100
##  1st Qu.:32.00   1st Qu.: 6500   1st Qu.:3.900
##  Median :40.00   Median : 7900   Median :4.700
##  Mean   :38.64   Mean   : 8309   Mean   :4.594
##  3rd Qu.:45.00   3rd Qu.: 9800   3rd Qu.:5.300
##  Max.   :54.00   Max.   :26400   Max.   :8.000
```

```
# data.imputed is not scaled/outlier_removed/log-trans/factorized
```

get confusion matrix

```
predicted <- ifelse(km$cluster == 2, 1, 0) # 2 = CKD
# Make factors with same levels
predicted <- factor(predicted, levels = c(0, 1))
actual <- factor(data.factorize$class, levels = c(0, 1))

cm.k <- confusionMatrix(data = predicted, reference = actual, positive = "1")
cm.k
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 148  71
##          1   0 162
##
##                Accuracy : 0.8136
##                  95% CI : (0.7709, 0.8515)
##     No Information Rate : 0.6115
```

```
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.6393
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.6953
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.6758
##              Prevalence : 0.6115
##          Detection Rate : 0.4252
##    Detection Prevalence : 0.4252
##       Balanced Accuracy : 0.8476
##
##        'Positive' Class : 1
##
```

get stats

```
accuracy <- cm.k$overall["Accuracy"]
sensitivity <- cm.k$byClass["Sensitivity"]
specificity <- cm.k$byClass["Specificity"]

cat("Accuracy: ", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy:  81.36 %
```

```
cat("Sensitivity: ", round(sensitivity * 100, 2), "%\n")
```

```
## Sensitivity:  69.53 %
```

```
cat("Specificity: ", round(specificity * 100, 2), "%\n")
```

```
## Specificity:  100 %
```

Conclusion for K-means clustering: It is good for predicting healthy patients but not sensitive enough to CKD patients.

2. hierarchical clustering

```
# remove column class
df<- subset(data.factorize, select = -class)

# 1. gower distance + clustering
d <- daisy(df, metric = "gower")
h <- hclust(d, method = "ward.D")
```

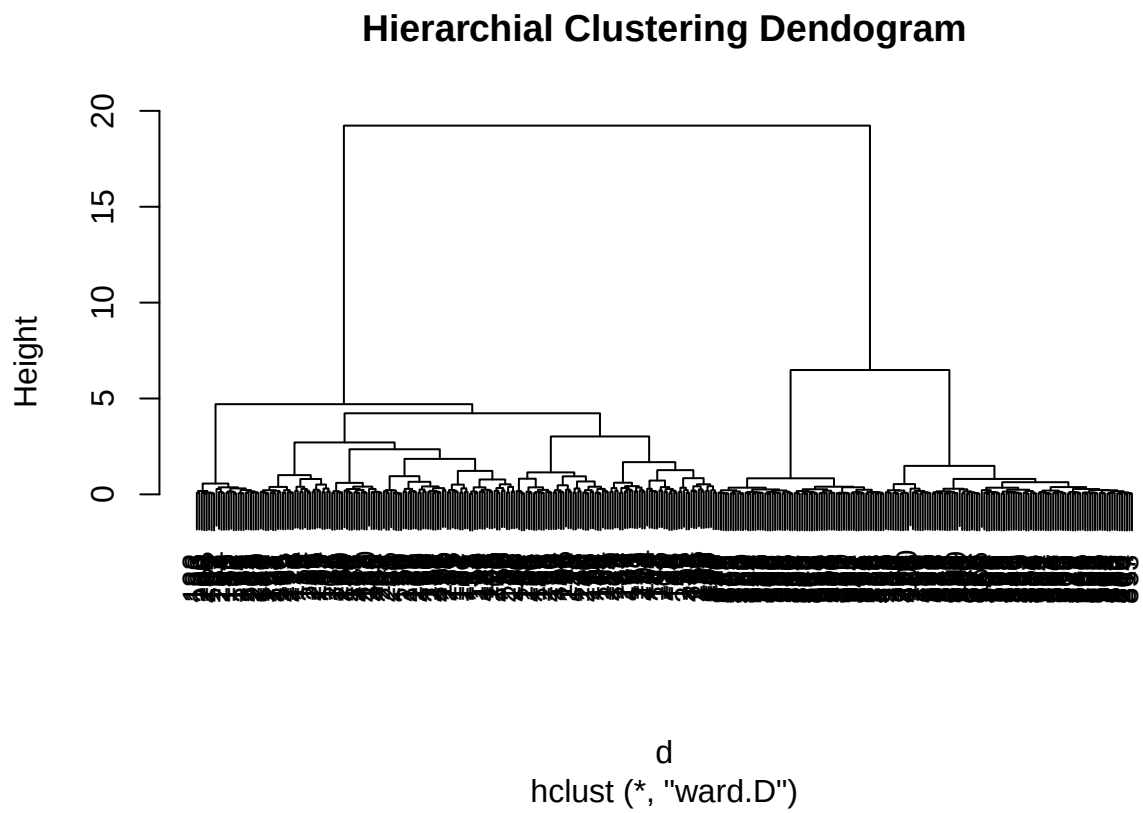
```r
cl <- factor(cutree(h, k = 2))

# testing which cluster is CKD
table(cl, data.factorize$class)
```

```
##
## cl    0   1
##   1   0 212
##   2 148  21
```

```r
# cluster 1 -> CKD  -> around 60
# cluster 2 -> not CKD   -> around 45


# temp df to combine data with new cluster column, for looking for centroids
df <- data.frame(df, cl = cl)
```

plot dendrogram

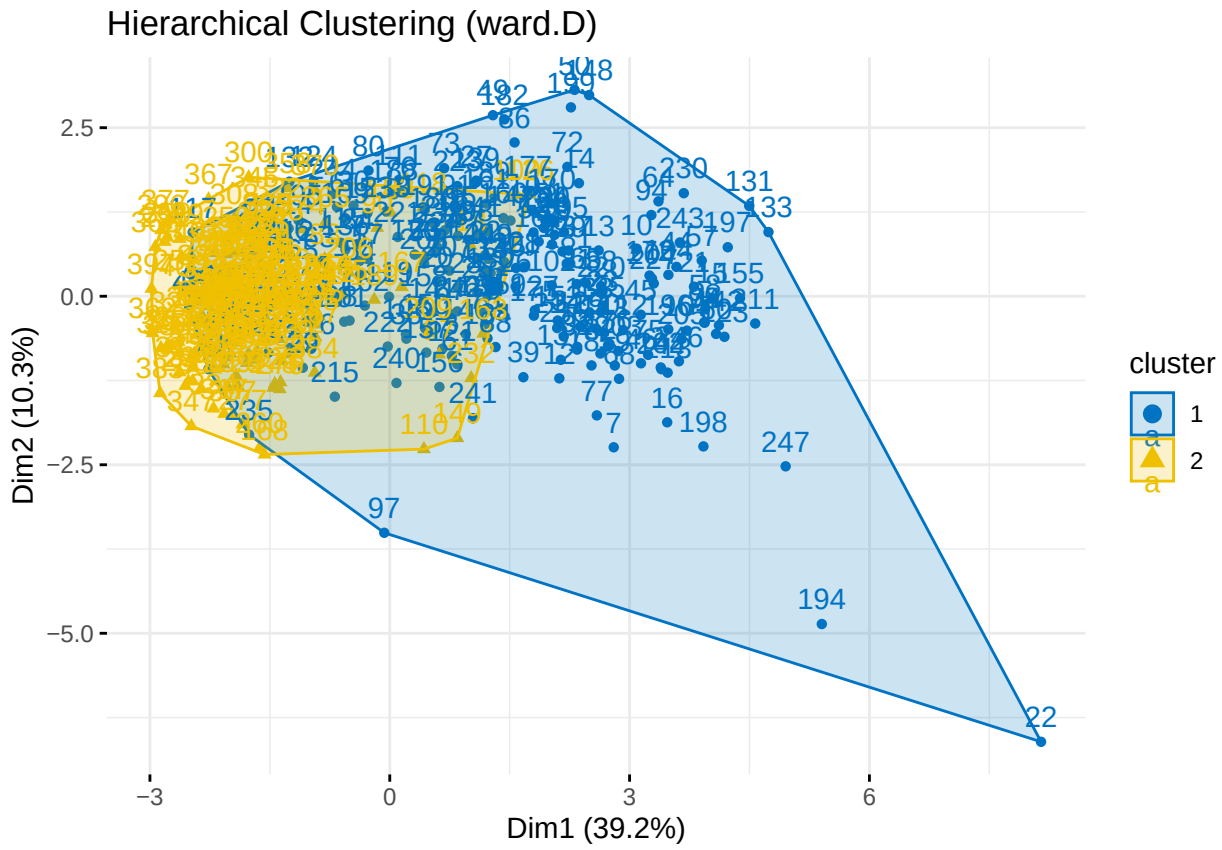
```r
plot(h, main='Hierarchial Clustering Dendogram', cex=0.9)
```



this is how we choose k=2.

plot clusters

```
# Plot using scaled numeric data and your cluster labels
fviz_cluster(list(data = scaled_numeric, cluster = cl),
             ellipse.type = "convex",
             palette = "jco",
             ggtheme = theme_minimal(),
             main = "Hierarchical Clustering (ward.D)")
```



get centroids

```
# 2. unscale + de-log numeric columns
num <- names(df)[sapply(df, is.numeric)]
cat <- setdiff(names(df), c(num, "cl"))

num_mean <- group_by(df, cl) %>% summarise(across(all_of(num), mean))

x <- as.matrix(num_mean[, -1])
x <- sweep(x, 2, sds[colnames(x)], "*")
x <- sweep(x, 2, means[colnames(x)], "+")
x[, skewed.cols] <- expm1(x[, skewed.cols])          # reverse log1p here

num_mean <- data.frame(cl = num_mean$cl, round(x, 2))
```

```r
# ------------------------------------------------------------------------------

# 3. summarize categorical data using mode
get_mode <- function(x) names(which.max(table(x)))
cat_mode <- group_by(df, cl) %>% summarise(across(all_of(cat), get_mode))

# ------------------------------------------------------------------------------

# 4. combine categorical & numeric centroids
centroids <- left_join(num_mean, cat_mode, by = "cl")
centroids
```

```
##   cl   age    bp    bgr    bu   sc    sod  pot  hemo   pcv rbcc   sg al su
## 1  1 55.23 78.69 147.43 54.05 2.75 135.36 4.38 11.08 34.18 4.09 1.01  0  0
## 2  2 47.13 71.31 111.19 31.66 1.00 140.93 4.27 14.84 45.49 5.31 1.02  0  0
```

```r
# remove temp variables
rm(d, h, df, num, cat, num_mean, cat_mode)
```

relabel centroids

```r
# notice that centroid orders are flipped, flip them back for better comparison
centroids$cl <- as.numeric(as.character(centroids$cl))
centroids$cl <- ifelse(centroids$cl == 1, 2, 1)

# reorder rows by new label
centroids <- centroids[order(centroids$cl), ]
rownames(centroids) <- NULL
centroids
```

```
##   cl   age    bp    bgr    bu   sc    sod  pot  hemo   pcv rbcc   sg al su
## 1  1 47.13 71.31 111.19 31.66 1.00 140.93 4.27 14.84 45.49 5.31 1.02  0  0
## 2  2 55.23 78.69 147.43 54.05 2.75 135.36 4.38 11.08 34.18 4.09 1.01  0  0
```

predict ckd or notckd

```r
# tested cluster 1 -> CKD
pred <- ifelse(cl == "1", "1", "0")
pred <- factor(pred, levels = c(0, 1))
actual <- factor(data.factorize$class, levels = c(0, 1))

cm.h <- confusionMatrix(pred, actual, positive = "1")
cm.h
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
```

```
##          0 148  21
##          1   0 212
##
##                Accuracy : 0.9449
##                  95% CI : (0.917, 0.9656)
##     No Information Rate : 0.6115
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.8869
##
##  Mcnemar's Test P-Value : 1.275e-05
##
##             Sensitivity : 0.9099
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.8757
##              Prevalence : 0.6115
##          Detection Rate : 0.5564
##    Detection Prevalence : 0.5564
##       Balanced Accuracy : 0.9549
##
##        'Positive' Class : 1
##
```

```r
# hierarchical clustering stats
accuracy <- cm.h$overall["Accuracy"]
sensitivity <- cm.h$byClass["Sensitivity"]
specificity <- cm.h$byClass["Specificity"]

cat("Accuracy: ", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy:  94.49 %
```

```r
cat("Sensitivity: ", round(sensitivity * 100, 2), "%\n")
```

```
## Sensitivity:  90.99 %
```

```r
cat("Specificity: ", round(specificity * 100, 2), "%\n")
```

```
## Specificity:  100 %
```

Conclusion for hierarchical clustering: Perfect predictions using method "ward.D" for hclust().

Other methods:

"ward.D2", gives 87.93% accuracy, 80.26% sensitivity, and 100% specificity.(flip)

"complete", gives 77.17% accuracy, 62.66% sensitivity, and 100% specificity.(not flip)

Overall, the hierarchical clustering is also at predicting non-CKD patients, and it is much more sensitive than k-means in case of CKD patients.

Research Question 3:

Can chronic kidney disease be accurately predicted when key predictors such as hemoglobin appear normal?

```
# cart model to check what variables are the most important
set.seed(597)
model_CART <- train( class~., data=train.data, method="rpart",
                     trControl = trainControl("cv",number=10), tuneLength=100)

# plot(model_CART)
# fancyRpartPlot(model_CART$finalModel)

# get CART model predictions
pred_CART <- predict(model_CART, newdata = test.data)

# ------------------------------------------------------------------------------

# check importance of each feature
model_CART$finalModel$variable.importance
```

```
##      hemo       pcv        sc      rbcc        bu       bgr   sg1.015       al2
## 92.481564 79.984055 77.384536 66.861671 53.741193 45.360190 19.326946  4.141489
##       pot       al4        bp       al1    sg1.01       su2
##  3.437768  2.760992  2.760992  2.291845  2.291845  1.380496
```

According to our research and exploration to the data, obtain new data with:

1. hemo within 12 - 16

2. pcv >= 36, everyone with pcv < 36 has ckd

3. sc <= 1.3, everyone with sc > 1.3 has ckd

4. rbcc >= 4, rbcc < 4 indicates anemia

```
# un-scale important features & get indices of target values
hemo_z1 <- (12 - means["hemo"]) / sds["hemo"]
hemo_z2 <- (16 - means["hemo"]) / sds["hemo"]
sc_z <- (1.3 - means["sc"]) / sds["sc"]
rbcc_z <- (4 - means["rbcc"]) / sds["rbcc"]
pcv_z <- (36 - means["pcv"]) / sds["pcv"]

new.data <- subset(data.factorize,
                   hemo >= hemo_z1 & hemo <= hemo_z2 &
                   sc <= sc_z &
                   rbcc >= rbcc_z &
                   pcv >= pcv_z)

# remove important features
```

```r
new.data <- new.data[, !(names(new.data) %in% c("hemo", "pcv", "sc", "rbcc"))]

nrow(new.data)          # 160 data left
```

```
## [1] 160
```

1. randomForest

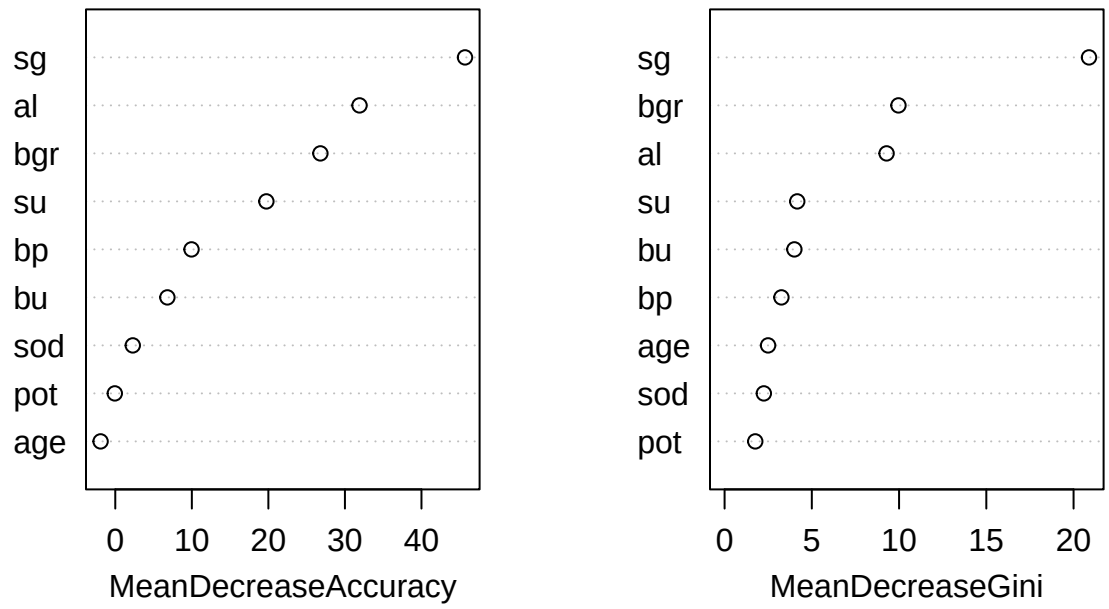
```r
# split data
set.seed(597)
training.samples <- new.data$class %>% createDataPartition(p=0.8, list=F)
train.data <- new.data[training.samples,]
test.data <- new.data[-training.samples,]
```

build model

```r
set.seed(597)
fit.rf.new <- randomForest(class ~ ., data = train.data, importance = TRUE, ntree = 500)
```

Plot variable importance

```r
varImpPlot(fit.rf.new, main = "Random Forest Feature Importance for CKD")
```



Get importance table

```
importance_df <- as.data.frame(fit.rf.new$importance)
importance_df <- importance_df[order(importance_df$MeanDecreaseGini, decreasing = TRUE), ]
print(importance_df)
```

```
##                 0            1 MeanDecreaseAccuracy MeanDecreaseGini
## sg   0.1730967383  0.124915667         1.544402e-01        20.885637
## bgr  0.0837837502  0.035472142         6.620479e-02         9.965090
## al   0.1026186111  0.061645166         8.734631e-02         9.284303
## su   0.0526734678  0.003650158         3.528807e-02         4.164864
## bu   0.0106358952  0.005173142         8.547063e-03         3.999468
## bp   0.0215720021  0.004820573         1.555398e-02         3.257533
## age -0.0020671733 -0.002198936        -2.202734e-03         2.493759
## sod  0.0003625237  0.007492803         2.589798e-03         2.249823
## pot -0.0014986416  0.002887525        -6.573285e-05         1.758278
```

```
rm(importance_df)
```

Predict model & analyze

```r
# get model predictions
rf_predictions <- predict(fit.rf.new, newdata = test.data)

# Create confusion matrix using caret
cm.f <- confusionMatrix(rf_predictions, test.data$class , positive="1")
cm.f
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 20  1
##          1  0 10
##
##                Accuracy : 0.9677
##                  95% CI : (0.833, 0.9992)
##     No Information Rate : 0.6452
##     P-Value [Acc > NIR] : 2.271e-05
##
##                   Kappa : 0.9281
##
##  Mcnemar's Test P-Value : 1
##
##             Sensitivity : 0.9091
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.9524
##              Prevalence : 0.3548
##          Detection Rate : 0.3226
##    Detection Prevalence : 0.3226
##       Balanced Accuracy : 0.9545
##
##        'Positive' Class : 1
##
```

Extract Metrics

```r
accuracy <- cm.f$overall["Accuracy"]
sensitivity <- cm.f$byClass["Sensitivity"]
specificity <- cm.f$byClass["Specificity"]

cat("Accuracy: ", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy:  96.77 %
```

```r
cat("Sensitivity: ", round(sensitivity * 100, 2), "%\n")
```

```
## Sensitivity:  90.91 %
```

```r
cat("Specificity: ", round(specificity * 100, 2), "%\n")
```

```
## Specificity:  100 %
```

Conclusion for RandomForest: A specificity of 100% indicates that the model is highly effective at correctly identifying non-CKD patients, even more so than its sensitivity of 90.91% for CKD patients. Overall, the model performs well. This suggests that even without the four most important features, the model can still effectively predict whether a patient has CKD or not.

2. Decision tree with 10-fold cross-validation and tuning

```r
set.seed(597)
fit.cart.new <- train(
  class ~ .,
  data = train.data,
  method = "rpart",
  trControl = trainControl(method = "cv", number = 10),
  tuneLength = 20
)

# Predict
pred.cart <- predict(fit.cart.new, newdata = test.data)
cm.c <- confusionMatrix(pred.cart, test.data$class, positive = "1")
cm.c
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##          0 20  3
##          1  0  8
##
##                Accuracy : 0.9032
##                  95% CI : (0.7425, 0.9796)
##     No Information Rate : 0.6452
##     P-Value [Acc > NIR] : 0.001141
##
##                   Kappa : 0.7748
##
##  Mcnemar's Test P-Value : 0.248213
##
##             Sensitivity : 0.7273
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 0.8696
##              Prevalence : 0.3548
##          Detection Rate : 0.2581
##    Detection Prevalence : 0.2581
##       Balanced Accuracy : 0.8636
##
```

```
##         'Positive' Class : 1
##
```

Extract Metrics

```
accuracy <- cm.c$overall["Accuracy"]
sensitivity <- cm.c$byClass["Sensitivity"]
specificity <- cm.c$byClass["Specificity"]

cat("Accuracy: ", round(accuracy * 100, 2), "%\n")
```

```
## Accuracy:  90.32 %
```

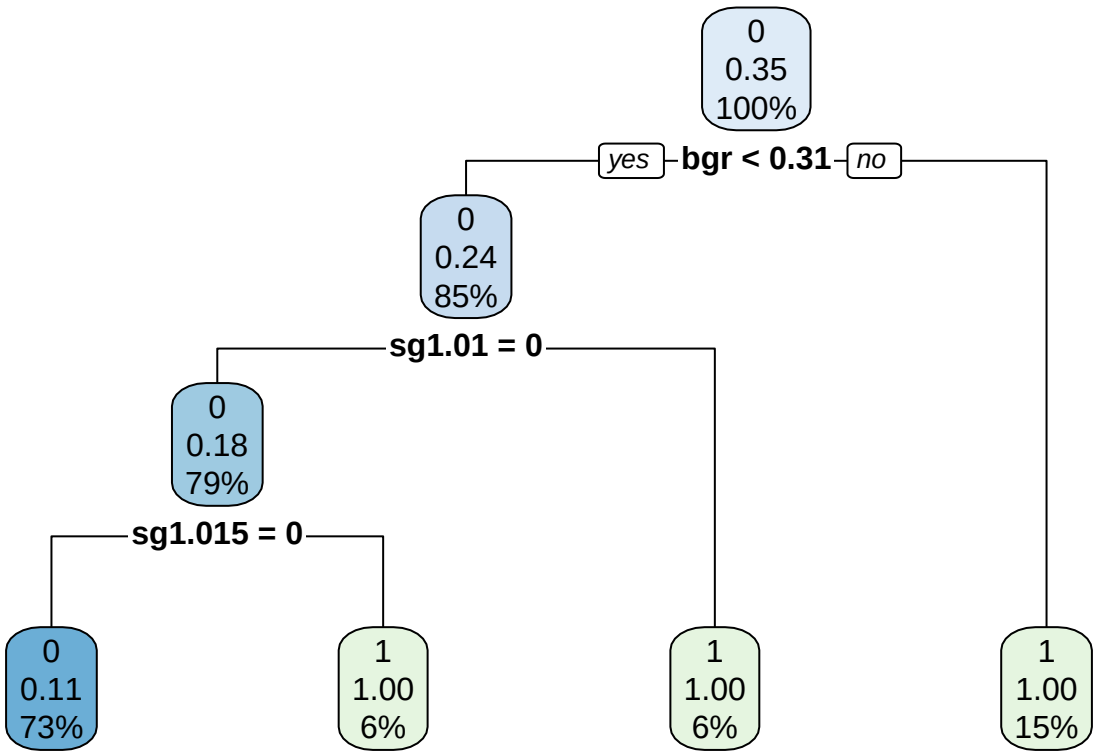
```
cat("Sensitivity: ", round(sensitivity * 100, 2), "%\n")
```

```
## Sensitivity:  72.73 %
```

```
cat("Specificity: ", round(specificity * 100, 2), "%\n")
```

```
## Specificity:  100 %
```

```
rpart.plot::rpart.plot(fit.cart.new$finalModel)
```



Conclusion for tuned CART model: Similar to the random forest model, the simpler tuned CART model built using the train() method also performs well in identifying healthy patients while achieving reasonable sensitivity in diagnosing CKD. This suggests that even when the strongest predictors appear normal, early signs of CKD can still be detected from secondary features.